

# UAV Mission Plan & Control Architecture

RobotX 2026 — Tasks 1–3 · ROS 2 **vision\_geo** (perception + geolocation) and **control** (flight behaviour)

## 1. System overview

Our UAV runs two ROS 2 nodes that work side by side. The first, **vision\_geo**, looks at the water through a downward-facing camera, runs a YOLO model to recognise the objects that matter in the current mission, and converts every detection from a pixel on the screen into a real GPS coordinate. The second, **control**, actually flies the aircraft: it arms and takes off, follows a survey pattern, holds a hover, centres the drone over a target, fires the servo that releases a payload, and brings the drone home. The two nodes are independent, so the camera keeps mapping while the drone keeps flying and neither one waits for the other. Everything that passes between our vehicles is plain ROS 2 topics: object positions travel as NavSatFix-style GPS messages, and each mission finishes with the UAV parking in a hover until the partner vehicle publishes the next "GO". We use one node binary and one YOLO model per mission — only the configuration file and the class list change between them.

## 2. The technology stack and what each piece does

At the bottom sits **ArduPilot with MAVROS**, flown in GUIDED mode. It executes the position and velocity setpoints we send it, and in return it gives us GPS, a local ENU pose, the attitude quaternion and the relative altitude — the four things the rest of the system depends on. Above that, **perception** is a YOLO model exported to ONNX and run through OpenCV's `cv::dnn` module at 320×320 or 640×640 input, producing a class id, a confidence and a bounding box for every object, with non-maximum suppression applied to remove duplicates.

The part that makes the whole system useful is **geolocation**. A bounding box on its own is just a location on a screen; the USV cannot navigate to it. So for every detection we take the pixel centre, turn it into a camera ray using the inverse camera matrix, rotate that ray into the drone body frame, then rotate it again into the world frame using the live attitude quaternion. Because we know the drone's height above the water, we intersect that ray with the flat water surface and get an East/North offset in metres, which we add to the drone's own position and convert into latitude and longitude. Using the real attitude quaternion, rather than assuming the drone is perfectly level, means that roll and pitch during the survey do not push the mapped position off target.

On top of that sits the **mapping** layer. A buoy seen in two hundred frames must not become two hundred map points, so detections are first matched frame to frame in pixel space, and a track only becomes a permanent map point once it has survived enough observations. From then on its stored GPS position is the running mean of every observation ever made of it, which means the position keeps getting more accurate on each pass instead of jumping around. The finished list leaves the node as a `vision_msgs/DetectedObjectArray` message carrying class id, label, confidence, pixel coordinates, latitude, longitude and the north/east offsets. On the flight side, the `control_` library gives us reusable building blocks — `takeoff`, `LocalMove`, `sendGlobalRaw`, `holdPosition`, `centering_object`, `controlServo`, `rotateByDegrees` and `setMode` — which a mission script simply calls in order. For monitoring, the node publishes RViz2 markers and TF alongside an annotated OpenCV window, so operators and judges can watch the map build up live.

## 3. Mission 1 — Safe Passage: the UAV maps the buoy field for the USV

In this task a field of buoys carries light beacons, and at the higher tiers the safe route through them is only visible from directly above — which means only the UAV can see it. The drone arms, takes off and climbs to a survey altitude of fifteen to twenty metres, then holds until its pose is stable. From there it flies a lawnmower pattern across the buoy field at constant altitude, keeping the camera level and the ground footprint predictable. While it flies, YOLO classifies each buoy and its beacon state — flashing red, flashing green, flashing blue for the entry point, solid blue for the exit, or off — and every accepted detection is ray-cast down to the water surface and converted to latitude and longitude. Repeated sightings of the same buoy merge into a single map point and re-average its position, so by the end of the pattern we hold one clean coordinate per buoy rather than a cloud of noisy guesses. The finished list, position plus colour, is published to the USV and to RoboCommand. The drone then climbs into a hover at its last survey point and waits: its job is done, but it stays on station in case the route has to be re-checked.

## 4. Mission 2 — Infrastructure Survey and Repair: the UAV delivers the resource tin

When the "GO" arrives, the drone leaves its hover and transits to the pipeline survey area. Two buoys mark the ends of the pipeline and only one of them shows a green colour indicator; the UAV detects which one it is and publishes its GPS so the UUV knows where to begin. It then returns to a hover and waits, because the next part of the task belongs to the UUV: it must survey the pipeline, find the damaged red segment and repair it. Once that is done the UUV sends the UAV a resource request — which colour tin is needed, and which coloured circle on the floating platform it must go to. The drone flies to the platform position that vision\_geo has already mapped, descends to drop altitude, and then switches from GPS navigation to visual servoing: it uses the pixel error between the image centre and the target circle to trim its position until it is genuinely centred over the circle, not merely near it. It fires the servo, confirms the release, reports completion, climbs back to a safe altitude and holds for the next mission.

## 5. Mission 3 — Coordinated Logistics: retrieve the correct tin and deliver it

This task starts with the USV, which must dock in the safe bay and put out the fire. When it has, it sends the UAV a two-colour request: the first colour is the tin that is needed, the second is the circle it must be delivered to. This is the hardest of the three for us, because the drone has to fetch the *correct* tin rather than any tin. The UAV transits to the source platform, detects the tins, and discards every detection whose class is not the requested colour. It centres over the chosen tin, descends and closes the gripper, confirms the pickup and climbs away. It then transits to the delivery platform, centres on the target colour circle exactly as in Mission 2, and releases. Finally it reports task completion to the USV and to RoboCommand, returns to launch and lands.

## 6. How the three missions are chained together

Every mission ends the same way. The UAV finishes its task, publishes its result, and then enters a hover-and-wait state — either at the point where it finished or at the staging point for the mission that follows. It leaves that state only when the partner vehicle publishes the mission "GO" topic. This makes the whole sequence event-driven rather than time-driven, and that distinction matters: if the USV takes longer to dock than we expected, or the UUV needs a second attempt at the pipeline, the UAV simply keeps hovering and nothing falls out of step. The same pattern also lets us re-run a single mission without restarting the entire system.

## 7. Robustness: what can go wrong and what we do about it

Most of the ways this system could fail are failures of *bad data* rather than crashes, so the guards are built into the pipeline itself. Every projection first checks that the GPS and pose are fresh — older than half a second and the frame is dropped rather than turned into a confidently wrong coordinate. A projection is also rejected if the drone is below half a metre above the water, or if the ground ray does not actually point downward, which prevents the division that would otherwise place an object at infinity. Against false and flickering detections, a track must survive five frames before it is allowed into the map, and an existing track is given a five-frame grace period so that a brief occlusion or a dropped frame does not destroy it. To stop one buoy being counted twice, a new map point merges into any existing point of the same class within two metres. Single-frame position noise is handled by averaging, as described above, and attitude-induced error by using the real quaternion on every ray.

On the flight side the principle is that no routine may hang. Every centering routine carries a timeout, and when it expires the drone falls back to a stable hover instead of drifting toward the water. The inference call has a fallback output path and throttled error logging, so a camera or model problem degrades into "no detections" while control carries on flying the planned pattern rather than the whole node dying. And behind all of it, `setMode("rtl")` is the terminal fallback in the mission script: whatever else fails, the drone always has a safe way home.

**Implementation status.** The vision\_geo perception and GPS mapping pipeline and the control function library are built and running. The current `src/control/src/mission/control.cpp` is still a bring-up script — take-off, a few fixed local moves, then RTL — so the per-mission sequences and the "GO" handshake described in sections 3 to 6 are the target design, and wiring them into that file is the next step. One item gates Mission 1 in particular: the model is currently configured with a single buoy class, so the beacon-colour classification that Task 1 depends on still has to be trained in.